



HikCentral Professional Video SDK (C#) Demo

User Manual

© 2021 Hangzhou Hikvision Digital Technology Co., Ltd. All rights reserved.

This Document (hereinafter referred to be "the Document") is the property of Hangzhou Hikvision Digital Technology Co., Ltd. or its affiliates (hereinafter referred to as "Hikvision"), and it cannot be reproduced, changed, translated, or distributed, partially or wholly, by any means, without the prior written permission of Hikvision. Unless otherwise expressly stated herein, Hikvision does not make any warranties, guarantees or representations, express or implied, regarding to the Document, any information contained herein.

LEGAL DISCLAIMER

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, THE DOCUMENT IS PROVIDED "AS IS" AND "WITH ALL FAULTS AND ERRORS". HIKVISION MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT. IN NO EVENT WILL HIKVISION BE LIABLE FOR ANY SPECIAL, CONSEQUENTIAL, INCIDENTAL, OR INDIRECT DAMAGES, INCLUDING, AMONG OTHERS, DAMAGES FOR LOSS OF BUSINESS PROFITS, BUSINESS INTERRUPTION OR LOSS OF DATA, CORRUPTION OF SYSTEMS, OR LOSS OF DOCUMENTATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, IN CONNECTION WITH THE USE OF THE DOCUMENT, EVEN IF HIKVISION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES OR LOSS.

Contents

Chapter 1	Overview	1
Chapter 2	Update History	2
Chapter 3	OpenAPI Test.....	3
Chapter 4	Live View	5
Chapter 5	Playback	8
Chapter 6	Two-Way Audio.....	13
Chapter 7	Event Receiving	14
Chapter 8	API Reference.....	15
7.1	CSHCVideoSDKImpl APIs	15
7.1.1	InitSDK	15
7.1.2	Video_StartPreview.....	15
7.1.3	Video_PlaySnap.....	16
7.1.4	Video_StartRecord	17
7.1.5	Video_GetVideoInfo	17
7.1.6	Video_StartPlayback	17
7.1.7	Video_StartPlayback_v2.....	18
7.1.8	Video_StartSingleFrame	19
7.1.9	Video_StartTalk.....	20
7.1.10	Video_PlayCtrl.....	20
7.1.11	Video_PlaybackPause.....	21
7.1.12	Video_PlaybackReplay	21
7.1.13	Video_PlaybackSeek.....	22
7.1.14	Video_PlaybackSpeed	22
7.1.15	Video_StartDownload	23
7.1.16	Video_StartDownload_v2.....	23
7.2	HttpClientImpl APIs.....	25
7.2.1	Class SignUtil	25
7.2.2	CreateRequest.....	25
7.2.3	initialBasicHeader.....	26
7.2.4	CheckValidationResult.....	26
7.2.5	GET	26
7.2.6	DELETE.....	27
7.2.7	POST	28
7.2.8	PUT.....	29

Chapter 1 Overview

The HikCentral Professional Video SDK Demo (hereinafter referred to as demo) is developed with C# language via Visual Studio 2015 and Net Framework 4.5.2. It provides API calling examples and the corresponding effects of OpenAPI test, live view, playback, two-way audio, and event receiving.

This demo has been configured with 32-bit and 64-bit compilation environment, and the executable file name is *CSHCSDKDemo.exe*. To bind the local IP address with the event receiving module, you should run the demo as an administrator.

Chapter 2 Update History

Summary of Changes in Version 3.0.0_Oct., 2023

- Added the functions of playback control, which support multiple time zone representations.
- Added the function of download control, which support multiple time zone representations.

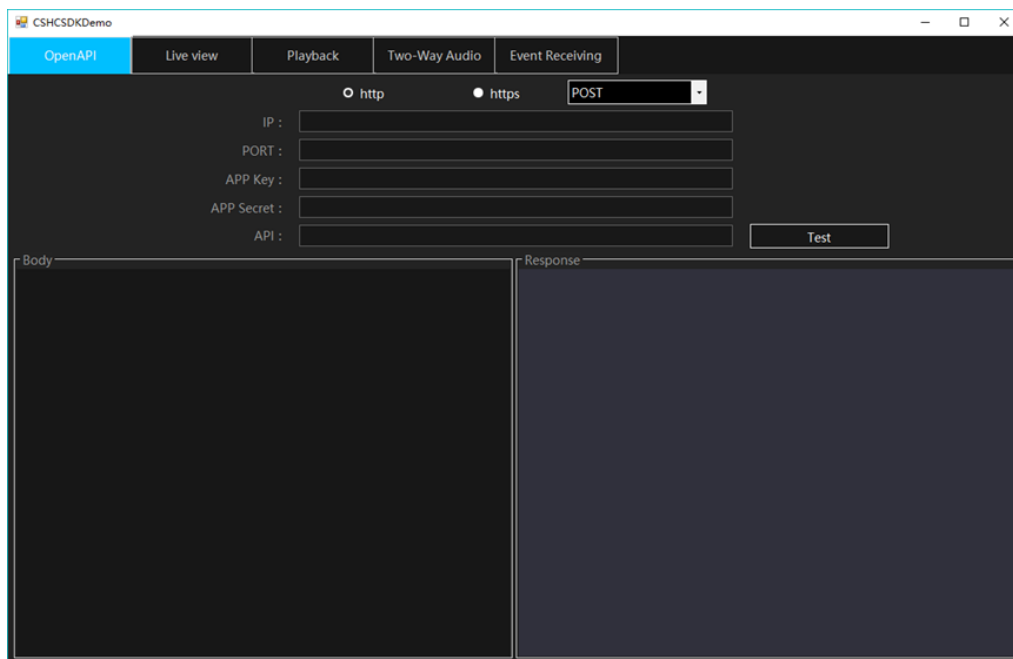
Chapter 3 OpenAPI Test

Purpose:

You can enter the gateway information and APIs to send test request and receive data from the gateway for checking the availability of OpenAPI.

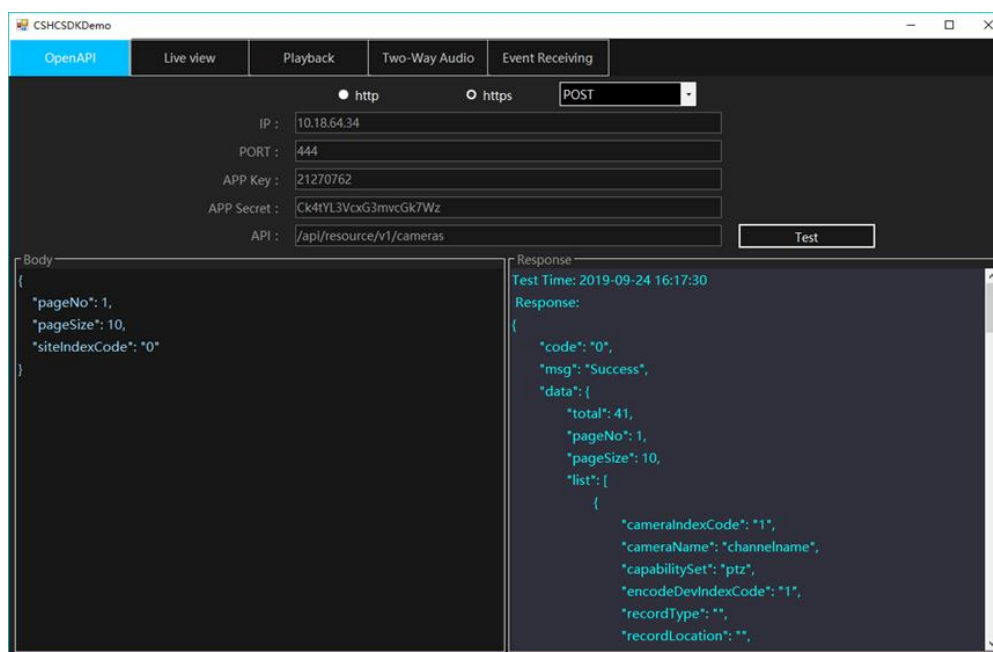
Steps:

1. Run the *CSHCSDKDemo.exe*.
2. Click **OpenAPI** tab to enter OpenAPI test page.



3. Configure the following parameters:
 - **Protocol type:** Whether to choose HTTP or HTTPS protocol.
 - **Request method:** Request method, including POST, GET, PUT, and DELETE.
 - **IP:** IP address of gateway.
 - **Port:** Port No. of gateway.
 - **APP Key:** Allocated APP key.
 - **APP Secret:** Allocated APP secret.
 - **API:** Request API.
 - **Body:** Protocol body.
4. Click **Test** to send the request.

After sending the request, the response data information will be displayed in Response area.

**Note:**

- The **Test Time** refers to the local time.
- For each test, the demo will automatically save the parameters including IP address, port No., APP key, APP secret, and API, etc.

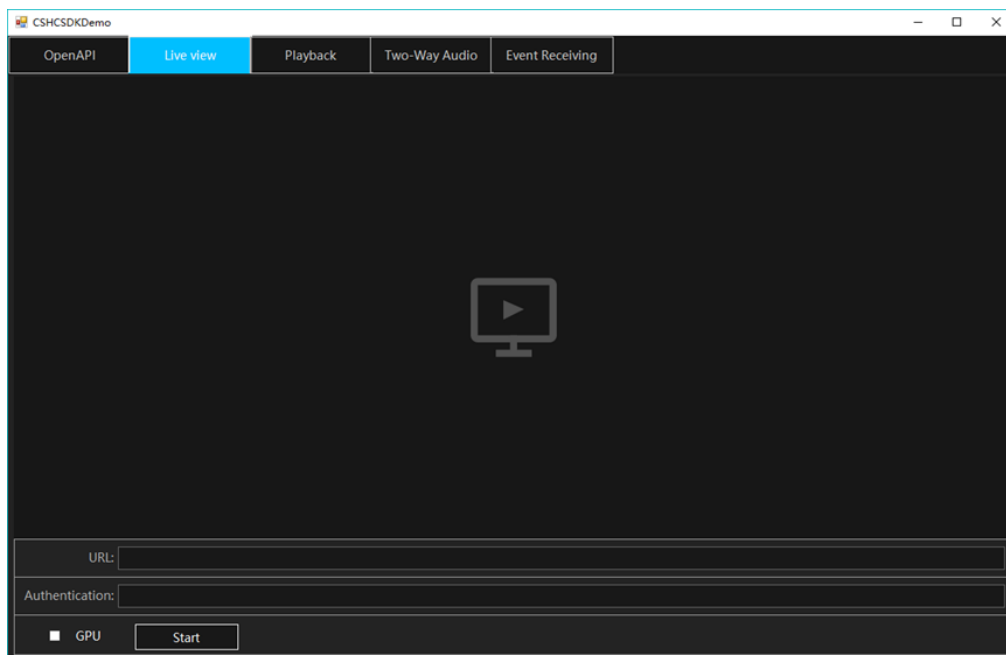
Chapter 4 Live View

Purpose:

You can enter the URL information to start live view, and then stop live view. Before starting live view, you can choose whether to enable GPU hardware decoding. During the live view, capturing, recording, viewing video details, and audio control are supported.

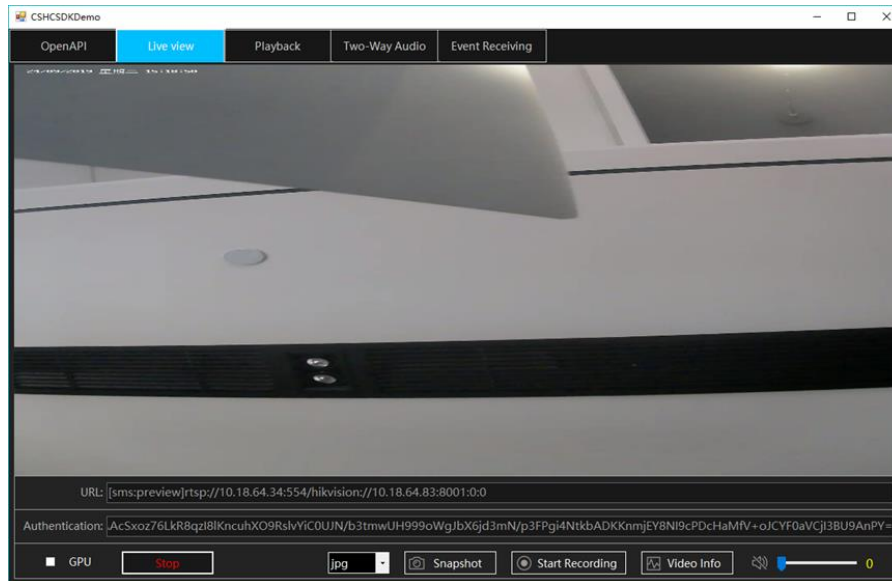
Steps:

1. Run the *CSHCSDKDemo.exe*.
2. Click **Live View** tab to enter Live View page.



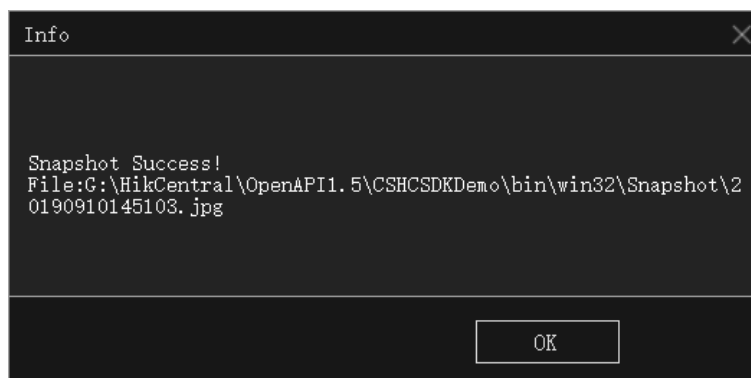
3. Configure the following parameters.
 - **URL:** Live view or streaming URL, which can be obtained by calling gateway API.
 - **Authentication:** authentication information.
 - **GPU:** Whether to enable GPU hardware decoding.
4. Click **Start** to start live view.

The button will turn to **Stop** in red color, and the live view function panel will be displayed in the lower-right corner.

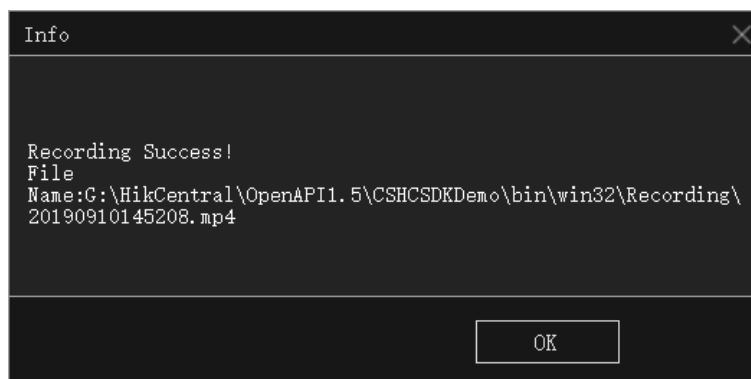


5. (Optional) Perform the following operation(s) after starting live view.

- **Capture:** You can select JPG or BMP picture in the drop-down list. Click **Snapshot** and select a saving path for picture in the pop-up window, then click **OK** to start capturing. After capturing, a window will appear to show the capture results, including failure or success information and the saving path of the captured picture.



- **Recording:** Click **Start Recording** and select a saving path for the video in the pop-up window, then click **OK** to start recording the current live video. Click **Stop Recording** to stop recording the video, and a window will appear to show the recording results, including failure or success information and the saving path of the recorded video. The recorded video must be played via Hikvision player.



- Video Details: Click **Video Info** to view the video details.



- Audio Control: Click  to turn on audio and click  to turn off audio. And you can drag on the volume bar to adjust the volume when the audio is on.



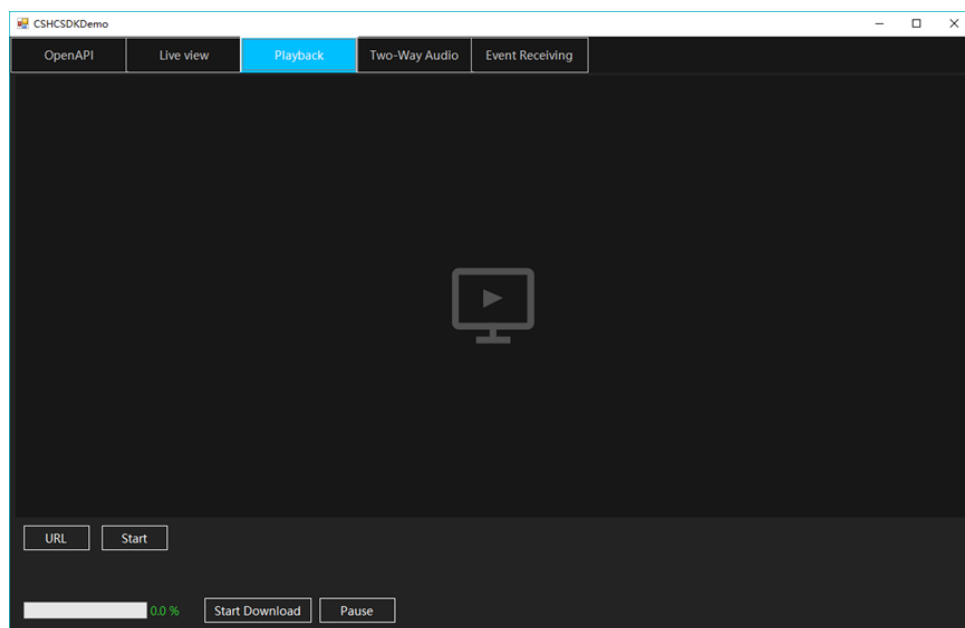
Chapter 5 Playback

Purpose:

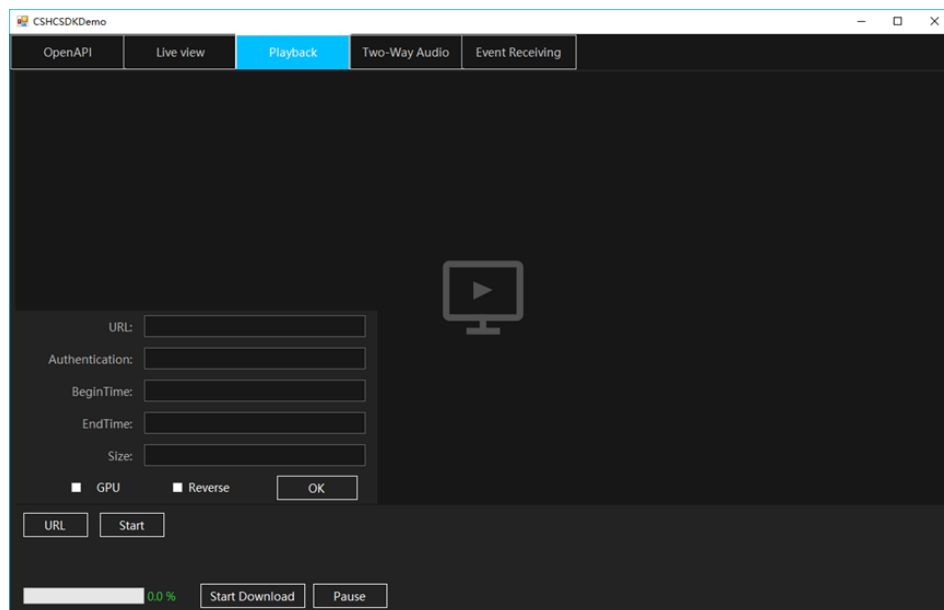
You can enter URL information and set start/end time to start playback or reverse playback. Before starting playback, you can choose whether to enable GPU hardware decoding. During playback, the following operations are supported, including pausing or resuming playback, capturing, viewing video details, audio control, located playback, fast/slow forward playback, single frame playback, downloading video, and so on.

Steps:

1. Run the *CSHCSDKDemo.exe*.
2. Click **Playback** tab to enter Playback page.

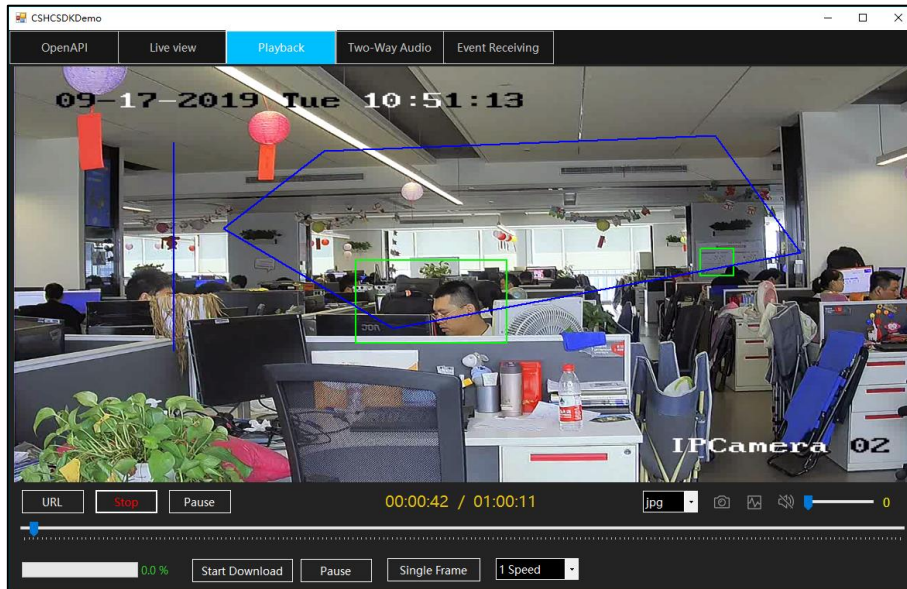


3. Click **URL** and configure the following parameters.

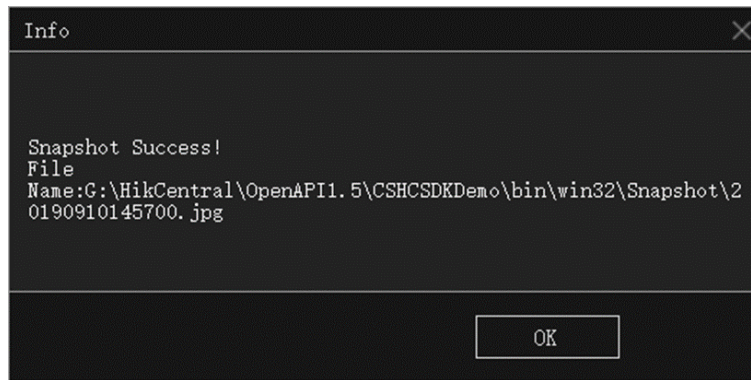


- **URL:** Playback or streaming URL, which can be obtained by calling gateway API.
 - **Authentication:** Authentication information.
 - **BeginTime:** Timestamp for starting playback (unit: second), which should be no earlier than the time for starting search for playback URL.
 - **EndTime:** Timestamp for stopping playback (unit: second), which should be no later than the time for stopping search for playback URL.
 - **Size:** Video segment size.
 - **GUP:** Whether to enable GUP hardware decoding.
 - **Reverse:** Whether to reverse playback.
4. Click **OK** to confirm the configuration.
 5. (Optional) Click **Start Downloading** to start downloading video file.
 6. Click **Start** to start playback.

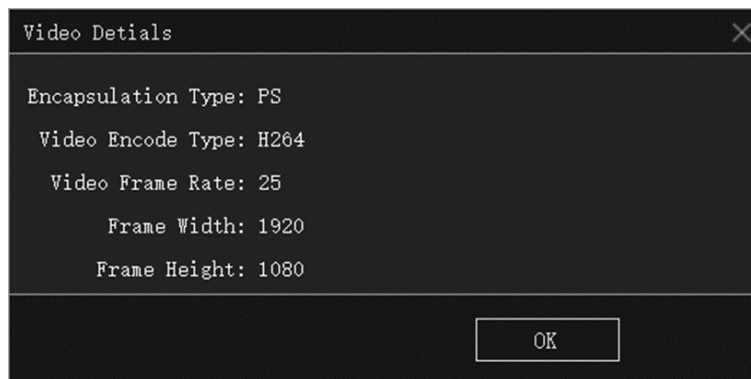
The button **Start** will change into **Stop** in red color. Total playback time, playback progress, and function panel will be displayed:



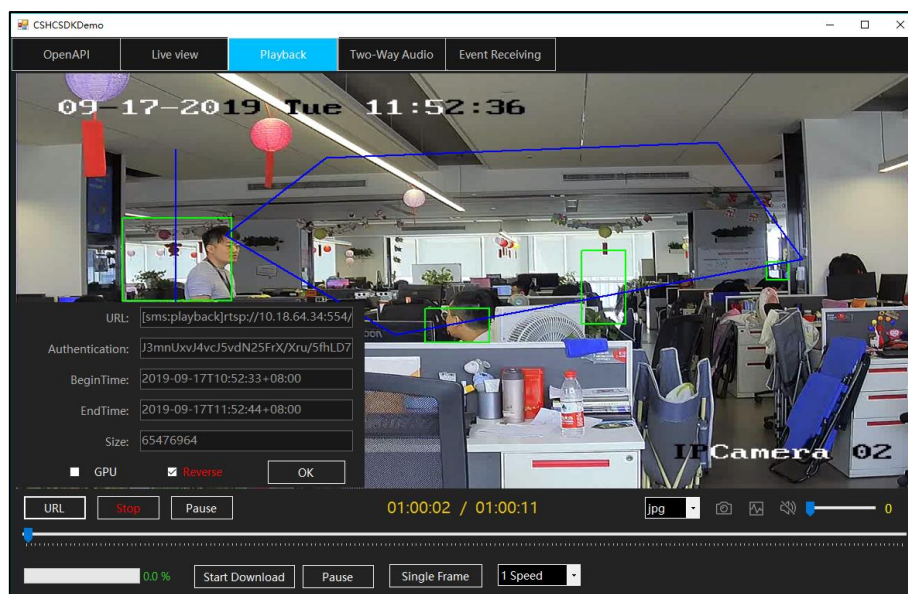
7. (Optional) Perform the following operation(s) after starting playback.
 - **Pause/Resume Playback:** Click **Pause** to pause the playback, and the button **Pause** will turn to **Restore**. Click **Restore** to resume playback.
 - **Capture:** You can select JPG or BMP picture in the drop-down list. Click  and select a saving path for picture in the pop-up window. Then click **OK** to start capturing picture. After capturing, a window will appear to show the capture results, including failure or success information and the saving path of the captured picture.



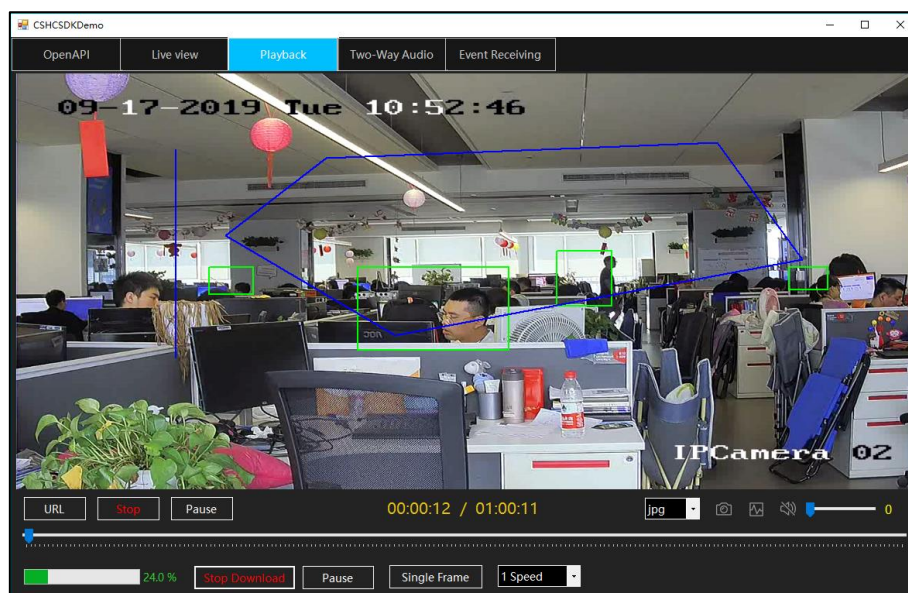
- Video Details: Click  to view the video details.



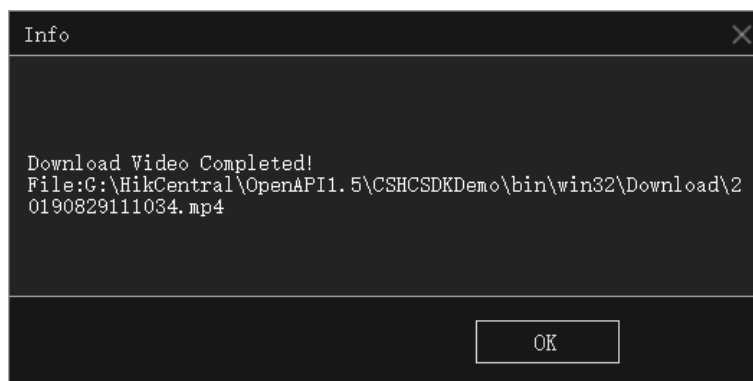
- Audio Control: Click  to turn on audio and click  to turn off audio. And you can drag on the volume bar to adjust the volume when the audio is on.
- Located Playback: Click on the playback time bar to specify a video time point to start playing, or select a date from calendar to play the videos of the day. The selected time point must be between the start time and end time of the playback video.
- Fast/Slow Forward Playback: Select the playback speed in the drop-down list for fast or slow forward playback.
- Reverse Playback: Check **Reverse** when configuring parameters in *step 3*, then click **Start** to start reverse playback in the specified time period. You can click **Pause** to stop reverse playback, and the button will turn to **Restore**. Click **Restore** to continue playing from the paused time point.



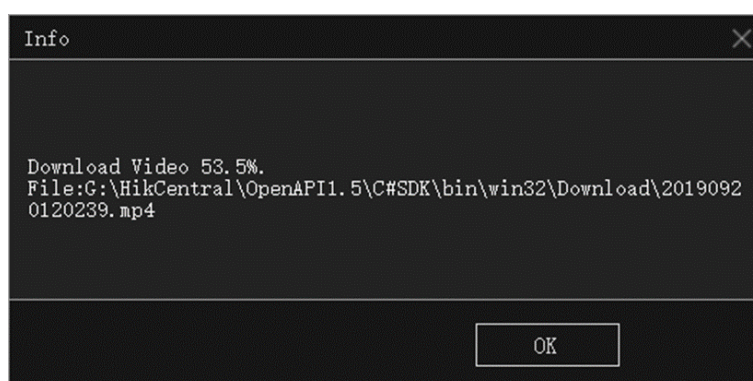
- Single Frame Playback: Click **Single Frame** to start single frame playback, and click the button again to view the next frame of the video.
- Download Video: Click **Start Download**, select a saving path for the video and click **OK** to start downloading the video according to the URL. The downloaded time duration is the same as that of playback, and the video size is exactly the size that has been configured previously.
- Pause/Resume Downloading: You can click **Pause** to stop downloading and the button will turn to **Recover**. Click **Recover** to resume downloading.



When downloading is completed, a window will appear to show the downloading results, including failure or success information and the saving path of the downloaded video.



If you click **Stop Download** during the process of downloading, a window will appear to show the downloading progress and the saving path of the downloaded video.



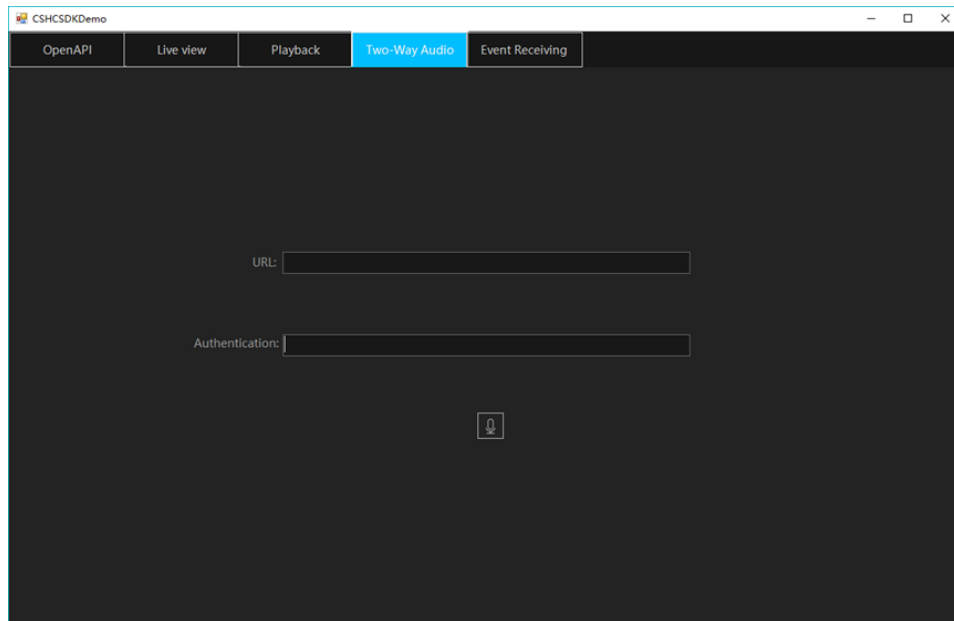
Chapter 6 Two-Way Audio

Purpose:

You can configure the URL parameters to start two-way audio with the device via the demo.

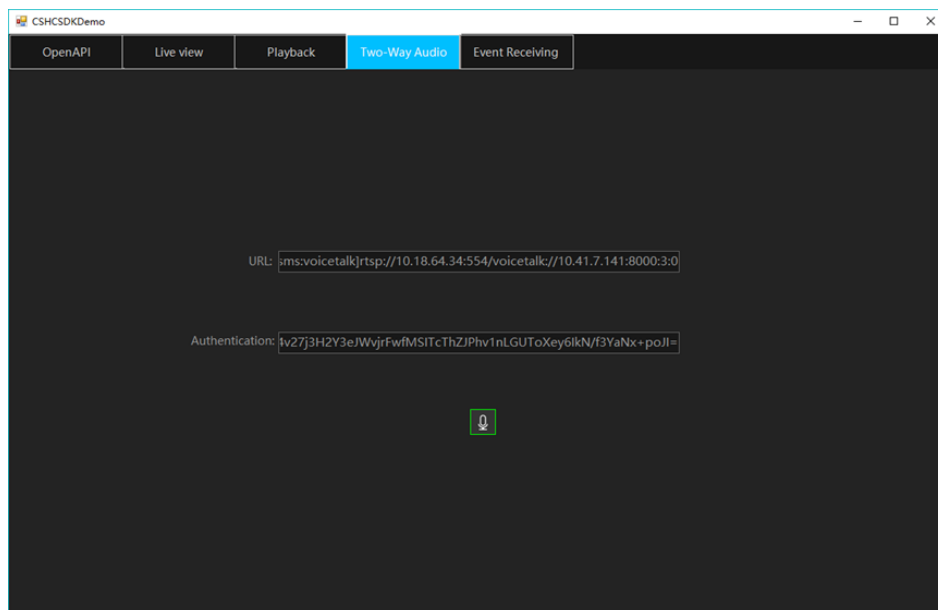
Steps:

1. Run the *CSHCSDKDemo.exe*.
2. Click **Two-Way Audio** tab to enter Two-Way Audio page.



3. Configure the following parameters.
 - **URL:** Two-way audio URL, which can be obtained from OpenAPI.
 - **Authentication:** Authentication information.
4. Click  to start two-way audio.

The color of  will turn to green after starting two-way audio; otherwise, it indicates that starting two-way audio failed.



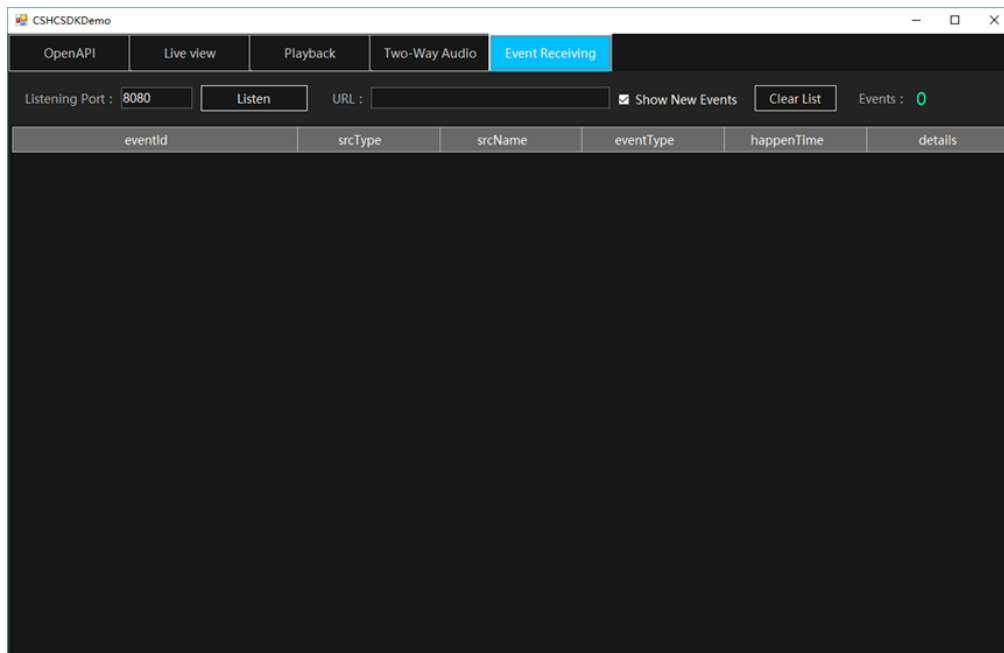
Chapter 7 Event Receiving

Purpose:

You can receive event information via the demo.

Steps:

1. Run the *CSHCSDKDemo.exe*.
2. Click **Event Receiving** tab to enter Event Receiving page.



3. Enter listening port No. and click **Listen** to start listening.
The button **Listen** will turn to **Listening** in green color, and the **URL** field will automatically be filled.
- Note:**
- The URL is the listening address of HTTP service, which should be copied to the subscription parameter as the event receiving address when subscribing events.
 - This demo does not support receiving event information via the listening address of HTTPS service.
4. (Optional) Perform the following operation(s) after starting listening.
 - **Show New Events:** whether to display the latest events.
 - **Clear List:** Clear the event list.
 - **View Event Source Code:** You can click **details** to view the event source code.
 - **View Total Number of Received Events:** The **Event** field shows the total number of received events in the list and the number will be updated automatically when new events are received.

Chapter 8 API Reference

7.1 CSHCVideoSDKImpl APIs

7.1.1 InitSDK

Description:

Initialize SDK, which should be called before starting running this demo.

Sample Code:

```
/// <summary>
/// Init SDK
/// </summary>
/// <returns></returns>
public int InitSDK()
{
    return HCVideoSDK.Video_Init("");
}
```

7.1.2 Video_StartPreview

Description:

Start live view.

Parameters:

- *play_info*: structure about video playing parameters.
- *open_gpu*: whether to enable GPU hardware decoding.
- *hWnd*: playing window handle.

Sample Code:

```

/// <summary>
/// Start Preview
/// </summary>
/// <param name="play_info">play info</param>
/// <param name="open_gpu">GPU Decode</param>
/// <param name="hWnd">window handle</param>
/// <returns></returns>
public int Video_StartPreview(PlayInfo play_info, bool open_gpu, IntPtr hWnd)
{
    stVideo_PlayParam param = new stVideo_PlayParam();
    param.iHardwareDecode = open_gpu ? 1 : 0;

    param.fnMsg = m_MsgCallback;
    param.fnStream = m_StreamCallback;
    param.fnDecodedStream = m_DecodeDataCallback;

    param.pUserData = StructToIntPtr((int)PlayType.PREVIEW);
    param.szReserve = new char[64];

    string pszUrl = MyJson.GetJsonStringByObject(play_info);
    return HCVideoSDK.Video_StartPreview(pszUrl, hWnd, StructToIntPtr(param));
}

```

Note:

- When converting the structure *play_info* into JSON format and transmitting it into SDK, structure *stVideo_PlayParam* should be created to configure callback function, configure GPU hardware decoding status, and set custom parameters.
- Enumeration type in *pUserData* can be used to identify whether it is the callback of live view or playback.
- Live view and playback can be distinguished by using different parameters in the same callback function, or by using different callback functions directly.

7.1.3 Video_PlaySnap

Description:

Capture picture during video playing.

Parameters:

- *iPlayHandle*: handle returned when starting playing video.
- *pszFileName*: absolute saving path of captured picture.

Sample Code:

```

/// <summary>
/// Snapshot
/// </summary>
/// <param name="iPlayHandle">video play handle</param>
/// <param name="pszFileName"></param>
/// <returns></returns>
public int Video_PlaySnap(int iPlayHandle, string pszFileName)
{
    return HCVideoSDK.Video_PlaySnap(iPlayHandle, pszFileName);
}

```

7.1.4 Video_StartRecord

Description:

Start recording video.

Parameters:

- *iPlayHandle*: handle returned after starting playing video.
- *pszFileName*: absolute saving path of MP4 video.

Sample Code:

```
/// <summary>
/// Preview Start Record
/// </summary>
/// <param name="iPlayHandle">start preview return handle</param>
/// <param name="pszFileName"></param>
/// <returns></returns>
public int Video_StartRecord(int iPlayHandle, string pszFileName)
{
    stVideo_RecordParam record_param = new stVideo_RecordParam();
    record_param.fnDownload = m_DownloadCallback;
    record_param.pUserData = IntPtr.Zero;
    record_param.lFileMaxSize = 1024 * 1024 * 1024;    //File Max Size:1GB

    return HCVideoSDK.Video_StartRecord(iPlayHandle, pszFileName, StructToIntPtr(record_param));
}
```

Note:

You can set the maximum size of a recorded video file. By default, it is set to 1G.

7.1.5 Video_GetVideoInfo

Description:

View video details.

Parameters:

- *iPlayHandle*: handle returned after starting playing video.
- *VideoDetailInfo*: video details.

Sample Code:

```
/// <summary>
/// Get Video Info
/// </summary>
/// <param name="iPlayHandle">video play handle</param>
/// <param name="VideoDetailInfo">return video detail info</param>
/// <returns></returns>
public int Video_GetVideoInfo(int iPlayHandle, ref stVideo_DetialInfo VideoDetailInfo)
{
    stVideo_DetialInfo video_info = new stVideo_DetialInfo();
    video_info.szReserve = new byte[64];
    IntPtr ptr = StructToIntPtr(video_info);
    int res = HCVideoSDK.Video_GetVideoInfo(iPlayHandle, ptr);
    VideoDetailInfo = IntPtrToStruct<stVideo_DetialInfo>(ptr);
    return res;
}
```

7.1.6 Video_StartPlayback

Description:

Start playback.

Parameters:

- *play_info*: playback parameters.
- *hWnd*: playback window handle.
- *iStartTimeStamp*: UTC timestamp of starting playback (unit: second).
- *iEndTimeStamp*: UTC timestamp of stopping playback (unit: second).
- *open_gpu*: whether to enable GPU hardware decoding.

Sample Code:

```

/// <summary>
/// Start Playback
/// </summary>
/// <param name="play_info">playback info</param>
/// <param name="hWnd">window handle</param>
/// <param name="iStartTimeStamp">start time</param>
/// <param name="iEndTimeStamp">end time</param>
/// <param name="open_gpu">GPU Decode</param>
/// <returns></returns>
public int Video_StartPlayback(
    PlayInfo play_info,
    IntPtr hWnd,
    int iStartTimeStamp,
    int iEndTimeStamp,
    bool open_gpu)
{
    stVideo_PlayParam param = new stVideo_PlayParam();
    param.iHardWareDecode = open_gpu? 1 : 0;

    param.fnMsg = m_MsgCallback;
    param.fnStream = m_StreamCallback;
    param.fnDecodedStream = m_DeCodeDataCallback;

    param.pUserData = StructToIntPtr((int)PlayType.PLAYBACK);
    param.szReserve = new char[64];

    string pszUrl = MyJson.GetJsonStringByObject(play_info);
    return HCVideoSDK.Video_StartPlayback(pszUrl, hWnd, iStartTimeStamp, iEndTimeStamp, StructToIntPtr(param));
}

```

Note:

- If *iStartTimeStamp* is earlier than *iEndTimeStamp*, it will start normal playback; If *iStartTimeStamp* is later than *iEndTimeStamp*, reverse playback will start; if *iStartTimeStamp* and *iEndTimeStamp* are the same, error will occur.
- When converting the structure *play_info* into JSON format and transmitting it into SDK, structure *stVideo_PlayParam* should be created to configure callback function, configure GPU hardware decoding status, and set custom parameters.
- Enumeration type in *pUserData* can be used to identify whether it is the callback of live view or playback.
- Live view and playback can be distinguished by using different parameters in the same callback function, or by using different callback functions directly.

7.1.7 Video_StartPlayback_v2

Description:

Start playback.

Parameters:

- *play_info*: playback parameters.
- *hWnd*: playback window handle.

- *iStartTimeStamp*: time string for starting playback, format: YYYY-MM-DDTHH: mm:xx. sssZ (e. g. 2023-09-18T01:00:00.000-08:00).
- *iEndTimeStamp*: time string for stopping playback, format: YYYY-MM-DDTHH: mm:xx. sssZ (e. g. 2023-09-18T01:00:00.000-08:00)
- *open_gpu*: whether to enable GPU hardware decoding.

Sample Code:

```

/// <summary>
    /// Start Playback
    /// </summary>
    /// <param name="play_info">playback info</param>
    /// <param name="hWnd">window handle</param>
    /// <param name="iStartTimeStamp">start time</param>
    /// <param name="iEndTimeStamp">end time</param>
    /// <param name="open_gpu">GPU Decode</param>
    /// <returns></returns>
    public int Video_StartPlayback(
        PlayInfo play_info,
        IntPtr hWnd,
        string iStartTimeStamp,
        string iEndTimeStamp,
        bool open_gpu)
    {
        stVideo_PlayParam param = new stVideo_PlayParam();
        param.iHardWareDecode = open_gpu ? 1 : 0;

        param.fnMsg = m_MsgCallback;
        param.fnStream = m_StreamCallback;
        param.fnDecodedStream = m_DecodeDataCallback;

        param.pUserData = StructToIntPtr((int)PlayType.PLAYBACK);
        param.szReserve = new char[64];

        string pszUrl = MyJson.GetJsonStringByObject(play_info);
        return HCVideoSDK.Video_StartPlayback_v2(pszUrl, hWnd, iStartTimeStamp, iEndTimeStamp,
        StructToIntPtr(param));
    }

```

Note:

- If *iStartTimeStamp* is earlier than *iEndTimeStamp*, it will start normal playback; If *iStartTimeStamp* is later than *iEndTimeStamp*, reverse playback will start; if *iStartTimeStamp* and *iEndTimeStamp* are the same, error will occur.
- When converting the structure *play_info* into JSON format and transmitting it into SDK, structure *stVideo_PlayParam* should be created to configure callback function, configure GPU hardware decoding status, and set custom parameters.
- Enumeration type in *pUserData* can be used to identify whether it is the callback of live view or playback.
- Live view and playback can be distinguished by using different parameters in the same callback function, or by using different callback functions directly.

7.1.8 Video_StartSingleFrame

Description:

Start single frame playback, and call this API again to view next frame.

Parameters:

iPlayHandle: handle returned after starting playing video.

Sample Code:

```
/// <summary>
/// Start SingleFrame
/// </summary>
/// <param name="iPlayHandle">video play handle</param>
/// <returns></returns>
public int Video_StartSingleFrame(int iPlayHandle)
{
    return HCVideoSDK.Video_StartSingleFrame(iPlayHandle);
}
```

7.1.9 Video_StartTalk

Description:

Start two-way audio. During two-way audio, the local audio information will be recorded, and by default, it will be saved in the folder *IntercomWav* under the program directory.

Parameters:

info: two-way audio information.

Sample Code:

```
/// <summary>
/// Start Talk
/// </summary>
/// <param name="info">Intercom info params</param>
/// <returns></returns>
public int Video_StartTalk(IntercomInfo info)
{
    //Intercom File Save Path
    string path = Environment.CurrentDirectory + "/IntercomWav/";
    if (false == System.IO.Directory.Exists(path))
    {
        System.IO.Directory.CreateDirectory(path);
    }
    string file_name = DateTime.Now.ToString("yyyyMMddHHmmss") + ".wav";
    string pszUrl = MyJson.GetJsonStringByObject(info);
    return HCVideoSDK.Video_StartTalk(pszUrl, path + file_name, m_MsgCallback, IntPtr.Zero);
}
```

7.1.10 Video_PlayCtrl

Description:

Control playback.

Parameters:

- *iPlayHandle*: video playing handle.
- *iCtrlType*: control types: 0-pause playback, 1-resume playback, 2-located playback, 3-control playback speed, 4-pause downloading, 5-resume downloading.
- *iParam*: control parameters, if *iCtrlType* is set to 2, *iParam* refers to UTC timestamp of located playback (unit: second); if *iCtrlType* is set to 3, *iParam* refers to playback speed, and the value can be set to 16, 8, 4, 2, 1, -2, -4, -8, -16 (the corresponding playback speeds in the speed drop-down

list on Playback page are 16, 8, 4, 2, 1, 1/2, 1/4, 1/8, 1/16, respectively); if *iCtrlType* is set to other values, the default value of *iParam* is 0.

Sample Code:

```
/// <summary>
/// Play Ctrl
/// </summary>
/// <param name="iPlayHandle">video play handle</param>
/// <param name="iCtrlType">CtrlType</param>
/// <param name="iParam">Param</param>
/// <returns></returns>
public int Video_PlayCtrl(int iPlayHandle, int iCtrlType, int iParam)
{
    return HCVideoSDK.Video_PlayCtrl(iPlayHandle, iCtrlType, iParam);
}
```

7.1.11 Video_PlaybackPause

Description:

Pause playback.

Parameters:

iPlayHandle: playback handle.

Sample Code:

```
/// <summary>
/// Playback Pause
/// </summary>
/// <param name="iPlayHandle">start playback return handle</param>
/// <returns></returns>
public int Video_PlaybackPause(int iPlayHandle)
{
    return HCVideoSDK.Video_PlayCtrl(iPlayHandle, (int)CtrlType.Pause, 0);
}
```

Note:

You can also set the value of *iCtrlType* to 0 in API [Video_PlayCtrl](#) to pause the playback.

7.1.12 Video_PlaybackReplay

Description:

Resume playback.

Parameters:

iPlayHandle: playback handle.

Sample Code:


```

/// <summary>
/// Playback Replay
/// </summary>
/// <param name="iPlayHandle">start playback return handle</param>
/// <returns></returns>
public int Video_PlaybackReplay(int iPlayHandle)
{
    return HCVideoSDK.Video_PlayCtrl(iPlayHandle, (int)CtrlType.ReplayPlayback, 0);
}

```

Note:

You can also set the value of *iCtrlType* to 1 in API [Video_PlayCtrl](#) to resume the playback.

7.1.13 Video_PlaybackSeek

Description:

Start located playback.

Parameters:

iPlayHandle: playback handle.

iTimeStamp: UTC timestamp of located playback (unit: second).

Sample Code:

```

/// <summary>
/// Playback Seek
/// </summary>
/// <param name="iPlayHandle">start playback return handle</param>
/// <param name="iTimeStamp">playback seek UTC timestamp</param>
/// <returns></returns>
public int Video_PlaybackSeek(int iPlayHandle, int iTimeStamp)
{
    return HCVideoSDK.Video_PlayCtrl(iPlayHandle, (int)CtrlType.PositionPlayback, iTimeStamp);
}

```

7.1.14 Video_PlaybackSpeed

Description:

Control playback speed.

Parameters:

- *iPlayHandle*: playback handle.
- *speed*: playback speed, the value can be set to 16, 8, 4, 2, 1, -2, -4, -8, -16, and the corresponding playback speeds in the speed drop-down list on playback page are 16, 8, 4, 2, 1, 1/2, 1/4, 1/8, 1/16, respectively.

Sample Code:

```

/// <summary>
/// Playback Speed
/// </summary>
/// <param name="iPlayHandle">start playback return handle</param>
/// <param name="speed">speed:1,2,4,8,16,-2,-4,-8,-16</param>
/// <returns></returns>
public int Video_PlaybackSpeed(int iPlayHandle, int speed)
{
    return HCVideoSDK.Video_PlayCtrl(iPlayHandle, (int)CtrlType.SpeedPlayback, speed);
}

```

7.1.15 Video_StartDownload

Description:

Start downloading the video.

Parameters:

- *play_info*: video parameters.
- *size*: size of the video file.
- *start_time*: timestamp for starting downloading.
- *end_time*: timestamp for stopping downloading.
- *file_name*: absolute saving path of downloaded video (including file name) in MP4 format.

Sample Code:

```

/// <summary>
/// Start Download
/// </summary>
/// <param name="play_info">play info</param>
/// <param name="size">video size</param>
/// <param name="start_time">start time</param>
/// <param name="end_time">end time</param>
/// <param name="file_name">file path</param>
/// <returns></returns>
public int Video_StartDownload(
    PlayInfo play_info,
    int size,
    int start_time,
    int end_time,
    string file_name)
{
    stVideo_DownloadParam param = new stVideo_DownloadParam();

    param.fnDownload = m_DownloadCallback;
    param.lStartTimeStamp = start_time;
    param.lEndTimeStamp = end_time;
    param.lMaxRecordSize = size;
    param.lFileMaxSize = size;

    param.pUserData = IntPtr.Zero;
    param.szReserve = new byte[64];

    string pszUrl = MyJson.GetJsonStringByObject(play_info);

    return HCVideoSDK.Video_StartDownload(pszUrl, file_name, StructToIntPtr(param));
}

```

Note:

Downloading progress can be obtained from the callback function *downloadCallback*.

7.1.16 Video_StartDownload_v2

Description:

Start downloading the video.

Parameters:

- *play_info*: video parameters.
- *size*: size of the video file.
- *start_time*: time string for starting downloading, format: *YYYY-MM-DDTHH:mm:ssZ* (e. g.

2023-09-18T01:00:00.000-08:00)

- *end_time*: time string for stopping downloading, format: YYYY-MM-DDTHH:mm:ssZ (e. g. 2023-09-18T01:00:00.000-08:00)
- *file_name*: absolute saving path of downloaded video (including file name) in MP4 format.

Sample Code:

```
/// <summary>
/// Start Download
/// </summary>
/// <param name="play_info">play info</param>
/// <param name="size">video size</param>
/// <param name="start_time">start time</param>
/// <param name="end_time">end time</param>
/// <param name="file_name">file path</param>
/// <returns></returns>
public int Video_StartDownload_v2(
    PlayInfo play_info,
    int size,
    string start_time,
    string end_time,
    string file_name)
{
    stVideo_DownloadParamEx param = new stVideo_DownloadParamEx();

    param.fnDownload = m_DownloadCallback;
    param.lStartTimeStamp = start_time;
    param.lEndTimeStamp = end_time;
    param.lMaxRecordSize = size;
    param.lFileMaxSize = 200 * 1024 * 1024;

    param.pUserData = IntPtr.Zero;
    param.szReserve = new byte[64];

    string pszUrl = MyJson.GetJsonStringByObject(play_info);

    return HCVideoSDK.Video_StartDownload_v2(pszUrl, file_name, StructToIntPtr(param));
}
```

7.2 HttpClientImpl APIs

7.2.1 Class SignUtil

Description:

This class is used to generate signature information.

7.2.2 CreateRequest

Description:

Send request.

Parameters:

- *url*: request URL.
- *body*: request parameters.
- *method*: request method, by default, the request method is GET.
- *headers*: request header.
- *timeout*: request timeout, by default, it is set to 30 s.
- *requestEncoding*: request encoding format, by default, the format is UTF-8.
- *AllowAutoRedirect*: whether the request supports URL redirection, by default, it is set to yes.

Sample Code:

```

/// <summary>
/// create HTTP/HTTPS request
/// </summary>
/// <param name="url">URL</param>
/// <param name="body">body</param>
/// <param name="timeout">request timeout</param>
/// <param name="userAgent">user agent</param>
/// <param name="requestEncoding">encode</param>
/// <param name="headers">http headers</param>
/// <param name="AllowAutoRedirect">Allow Request Auto Redirect</param>
/// <returns></returns>
private static HttpWebResponse CreateRequest(
    string url,
    string body = null,
    string method = HTTP_GET,
    string headers = null,
    int timeout = 30,
    Encoding requestEncoding = null,
    bool AllowAutoRedirect = true)
{
    if (string.IsNullOrEmpty(url))
        throw new ArgumentNullException("url");

    if (requestEncoding == null)
        requestEncoding = Encoding.GetEncoding("UTF-8");

    HttpWebRequest request = null;

    //https request need set SecurityProtocol
    if (url.StartsWith("https", StringComparison.OrdinalIgnoreCase))
    {
        ServicePointManager.SecurityProtocol = SecurityProtocolType.Tls12;
        ServicePointManager.ServerCertificateValidationCallback = new RemoteCertificateValidationCallback(CheckValidationResult);
        request = WebRequest.Create(url) as HttpWebRequest;
        request.ProtocolVersion = HttpVersion.Version10;
    }
}

```

Note:

For HTTPS request, you should set the security protocol type of the transport layer (*ServicePointManager.SecurityProtocol*). In this sample, the protocol type is TLS 1.2.

7.2.3 initialBasicHeader

Description:

Generate HTTP request header, and the request header will be encrypted by the secret key and encryption method.

Parameters:

- *header*: generated request header, which can be encrypted by secret key and encryption method.
- *appKey*: APP key provided by gateway.
- *appSecret*: APP secret provided by gateway.
- *method*: request method.
- *requestAddress*: request URL.
- *formParam*: form parameters.
- *signHeaderPrefixList*: extra prefix list that should be added to the signature information.

Sample Code:

```
/// <summary>
/// create http headers and sign in info
/// </summary>
/// <param name="header"></param>
/// <param name="appKey"></param>
/// <param name="appSecret"></param>
/// <param name="method"></param>
/// <param name="requestAddress"></param>
/// <param name="formParam"></param>
/// <param name="signHeaderPrefixList"></param>
private static void initialBasicHeader(
    ref Dictionary<string, string> header,
    string appKey,
    string appSecret,
    string method,
    string requestAddress,
    Dictionary<string, string> formParam,
    List<string> signHeaderPrefixList)
{
    header.Add(X_CA_KEY, appKey);
    header.Add(X_CA_NONCE, GetGUIDString());
    header.Add(X_CA_TIMESTAMP, GetUTCTimeStamp());
    header.Add(X_CA_SIGNATURE, SignUtil.Sign(method, requestAddress, ref header, formParam, appSecret, signHeaderPrefixList));
}
```

7.2.4 CheckValidationResult

Description:

Verify certificate. This function is not supported and all the certificates are valid by default.

Sample Code:

```
/// <summary>
/// Check Certificate
/// </summary>
/// <param name="sender"></param>
/// <param name="certificate"></param>
/// <param name="chain"></param>
/// <param name="errors"></param>
/// <returns></returns>
private static bool CheckValidationResult(object sender, X509Certificate certificate, X509Chain chain, SslPolicyErrors errors)
{
    return true; //receive all
}
```

7.2.5 GET

Description:

Send request by GET method.

Parameters:

- *url*: request URL.
- *headers*: request headers.
- *appKey*: APP key provided by gateway.
- *appSecret*: APP secret provided by gateway.
- *timeout*: timeout, by default it is set to 10 s.

Sample Code:

```
public static string Get(
    string url,
    Dictionary<string, string> headers,
    string appKey,
    string appSecret,
    int timeout = 10)
{
    List<string> signHeaderPrefixList = new List<string>();
    Dictionary<string, string> formParam = null;
    initialBasicHeader(ref headers, appKey, appSecret, HTTP_GET, url, formParam, signHeaderPrefixList);

    string header_string = "";
    foreach (var kvp in headers)
    {
        header_string += (kvp.Key + ":" + kvp.Value + "\n");
    }

    string content = "";
    try
    {
        HttpWebResponse rq = CreateRequest(url, null, HTTP_GET, header_string, timeout * 1000);
        Stream rs = rq.GetResponseStream();
        HttpStatusCode http_Code = rq.StatusCode;
        using (StreamReader sr = new StreamReader(rs, Encoding.UTF8))
        {
            content = sr.ReadToEnd();
        }
    }
    catch (Exception e)
    {
        content = e.Message;
    }
    return content;
}
```

7.2.6 DELETE

Description:

Send request by DELETE method.

Parameters:

- *url*: request URL.
- *headers*: request headers.
- *appKey*: APP key provided by gateway.
- *appSecret*: APP secret provided by gateway.
- *timeout*: timeout, by default it is set to 10 s.

Sample Code:

```

public static string Delete(
    string url,
    Dictionary<string, string> headers,
    string appKey,
    string appSecret,
    int timeout = 10)
{
    List<string> signHeaderPrefixList = new List<string>();
    Dictionary<string, string> formParam = null;
    initialBasicHeader(ref headers, appKey, appSecret, HTTP_DELETE, url, formParam, signHeaderPrefixList);

    string header_string = "";
    foreach (var kvp in headers)
    {
        header_string += (kvp.Key + ":" + kvp.Value + "\n");
    }

    string content = "";
    try
    {
        HttpWebResponse rq = CreateRequest(url, null, HTTP_DELETE, header_string, timeout * 1000);
        Stream rs = rq.GetResponseStream();
        HttpStatusCode http_Code = rq.StatusCode;
        using (StreamReader sr = new StreamReader(rs, Encoding.UTF8))
        {
            content = sr.ReadToEnd();
        }
    }
    catch (Exception e)
    {
        content = e.Message;
    }
    return content;
}

```

7.2.7 POST

Description:

Send request by POST method.

Parameters:

- *url*: request URL.
- *headers*: request headers.
- *appKey*: APP key provided by gateway.
- *appSecret*: APP secret provided by gateway.
- *body*: request parameters.
- *timeout*: timeout, by default it is set to 10 s.

Sample Code:

```

public static string Post(
    string url,
    Dictionary<string, string> headers,
    string appKey,
    string appSecret,
    string body = null,
    int timeout = 10)
{
    List<string> signHeaderPrefixList = new List<string>();
    Dictionary<string, string> formParam = null;
    initialBasicHeader(ref headers, appKey, appSecret, HTTP_POST, url, formParam, signHeaderPrefixList);

    string header_string = "";
    foreach (var kvp in headers)
    {
        header_string += (kvp.Key + ":" + kvp.Value + "\n");
    }

    string content = "";
    try
    {
        HttpWebResponse rq = CreateRequest(url, body, HTTP_POST, header_string, timeout * 1000);
        Stream rs = rq.GetResponseStream();
        HttpStatusCode http_Code = rq.StatusCode;
        using (StreamReader sr = new StreamReader(rs, Encoding.UTF8))
        {
            content = sr.ReadToEnd();
        }
    }
    catch (Exception e)
    {
        content = e.Message;
    }
    return content;
}

```

7.2.8 PUT

Description:

YSend request by PUT method.

Parameters:

- *url*: request URL.
- *headers*: request headers.
- *appKey*: APP key provided by gateway.
- *appSecret*: APP secret provided by gateway.
- *body*: request parameters.
- *timeout*: timeout, by default it is set to 10 s.

Sample Code:


```
public static string Put(
    string url,
    Dictionary<string, string> headers,
    string appKey,
    string appSecret,
    string body = null,
    int timeout = 10)
{
    List<string> signHeaderPrefixList = new List<string>();
    Dictionary<string, string> formParam = null;
    initialBasicHeader(ref headers, appKey, appSecret, HTTP_PUT, url, formParam, signHeaderPrefixList);

    string header_string = "";
    foreach (var kvp in headers)
    {
        header_string += (kvp.Key + ":" + kvp.Value + "\n");
    }

    string content = "";
    try
    {
        HttpWebResponse rq = CreateRequest(url, body, HTTP_PUT, header_string, timeout * 1000);
        Stream rs = rq.GetResponseStream();
        HttpStatusCode http_Code = rq.StatusCode;
        using (StreamReader sr = new StreamReader(rs, Encoding.UTF8))
        {
            content = sr.ReadToEnd();
        }
    }
    catch (Exception e)
    {
        content = e.Message;
    }
    return content;
}
```



See Far, Go Further